

Laboratório de Experimentação de Software

Lab01 — Características de Repositórios Populares

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab01 — Características de Repositórios Populares
Grupo (trio)	Bernardo Souza Alvim, Luísa Oliveira Jardim, Pedro Augusto Santos Seabra
Link do repositório / GitHub Projects	https://github.com/luisajardim/Lab-Experimentacao-e-Medicao-
Data de entrega	28/08/2026

1. Introdução

Este laboratório investiga se repositórios populares do GitHub (os 1.000 com mais estrelas) compartilham certas características: idade, contribuição externa, frequência de releases, frequência de atualização, linguagem de programação e percentual de issues fechadas. A árvore GQM (Goal-Question-Metric) usada para derivar as métricas de cada Questão de Pesquisa (RQ) a partir desses objetivos está detalhada na Tabela de Métricas (seção 3.5).

Para cada uma das 7 RQs do enunciado, o grupo registrou uma hipótese informal antes de olhar qualquer dado coletado. Essas hipóteses são listadas abaixo; o confronto de cada uma com o resultado obtido no dataset final está na seção 4.3 (Discussão).

Questões de Pesquisa e hipóteses informais

RQ01 — Sistemas populares são maduros/antigos?

A hipótese estabelece que a popularidade é um atributo construído ao longo do tempo. Espera-se que a maioria dos repositórios com alto volume de estrelas possua maturidade elevada, com baixa incidência de projetos com menos de dois anos e concentração predominante na faixa de 5 a 15 anos.

RQ02 — Sistemas populares recebem muita contribuição externa?

Sim. Repositórios com muitas estrelas tendem a ter uma comunidade maior e mais visibilidade, o que deve atrair contribuições na forma de pull requests. Esperamos que a maioria possua uma quantidade relevante de PRs aceitas, embora alguns projetos muito grandes possam concentrar valores excepcionalmente altos.

RQ03 — Sistemas populares lançam releases com frequência?

Sim, para uma parcela expressiva. Projetos populares normalmente precisam comunicar versões, correções e novas funcionalidades. Porém, espera-se também que alguns repositórios não usem a funcionalidade Releases do GitHub (ex.: distribuição por tags, pacotes ou imagens de container).

RQ04 — Sistemas populares são atualizados com frequência?

Estima-se que a visibilidade incentive a manutenção contínua. Contudo, prevê-se a existência de um subconjunto de repositórios que preservam sua popularidade apesar da ausência de atualizações recentes, como repositórios de curadoria (listas awesome-*) ou ferramentas maduras em estado de manutenção mínima.

RQ05 — Sistemas populares são escritos nas linguagens mais populares?

Sim. Usando o GitHub Octoverse 2025 como referência única de popularidade de linguagens (TypeScript, Python e JavaScript no topo), esperamos que a maior parte dos repositórios mais estrelados tenha uma dessas linguagens como `primaryLanguage`.

RQ06 — Sistemas populares possuem alto percentual de issues fechadas?

Sim. Projetos com muitas estrelas tendem a ter comunidade e automação suficientes para triagem contínua. Esperamos mediana da razão issues fechadas / total de issues acima de 0,7 (70%). Repositórios sem aba de Issues entram com razão 0 e não são excluídos.

RQ07 — Sistemas em linguagens mais populares recebem mais contribuição, releases e atualização?

Sim. Esperamos medianas maiores de PRs aceitas (RQ02) e de releases (RQ03), e mediana menor de dias desde o último push (RQ04), nos repositórios cuja linguagem está no top do Octoverse 2025, em comparação com as demais linguagens da amostra.

O projeto usou uma arquitetura análoga à hexagonal, separando script de coleta, especificações e orquestrador, para tornar a mineração de dados genérica.

2. Contexto

Este é o Lab01 da disciplina, o laboratório inicial de mineração de dados do semestre. O objeto de estudo são os 1.000 repositórios mais estrelados do GitHub, caracterizados quanto a idade, contribuição externa (pull requests aceitas), frequência de releases, frequência de atualização (pushes), linguagem de programação primária e percentual de issues fechadas.

Como base teórica para a derivação de métricas a partir dos objetivos do estudo, o grupo utilizou o método GQM (Goal-Question-Metric) de Basili, Caldiera & Rombach — a árvore e a tabela GQM completas estão na seção 3.5. Para a questão de "linguagens mais populares" (RQ05/RQ07), a fonte de referência adotada — e mantida como única do início ao fim do laboratório — foi o GitHub Octoverse 2025 (ranking por contribuidores, agosto/2025), com destaque para TypeScript, Python, JavaScript, Java e C# no topo.

3. Metodologia

3.1 Principais Desafios

Um dos primeiros desafios foi lidar com o limite de requisições da API GraphQL do GitHub, já que precisávamos consultar milhares de repositórios. Para evitar que o processo parasse no meio, foi necessário cuidar da paginação e do tratamento de erros no script de mineração (Miner).

Também tivemos dificuldades para padronizar os dados. Alguns repositórios não possuem releases, issues habilitadas ou uma linguagem principal definida. Por isso, tivemos que definir algumas regras para esses casos. No RQ06, por exemplo, decidimos manter os repositórios sem issues na amostra e considerar a razão como 0, em vez de excluí-los. Da mesma forma, foi importante diferenciar um repositório que realmente não possui releases de um dado que estava faltando, para não alterar os resultados das medianas e dos percentuais.

Por fim, a quantidade de dados também trouxe um desafio para dividir o trabalho entre os três integrantes. Cada um ficou responsável por algumas partes da extração (RQ01/RQ04, RQ02/RQ03 e RQ05/RQ06/RQ07). Antes de juntar tudo em um único CSV, cada parte foi conferida separadamente para evitar erros e retrabalho durante a análise.

3.2 Tomadas de Decisão

- Limite de WIP no board: 3 cartões na coluna "Em Andamento", para reduzir multitarefa, expor bloqueios mais cedo e manter o fluxo de sprint mais previsível.
- Fonte de popularidade de linguagens (RQ05/RQ07): GitHub Octoverse 2025 foi escolhida como fonte única, mantida do início ao fim do laboratório, em vez de combinar múltiplos rankings.
- RQ06 — repositórios com `total_issues = 0` recebem `ratio_closed_issues = 0` e são mantidos na amostra (não excluídos), para não inflar artificialmente a mediana.

- Padronização do CSV: valores nulos, ausentes ou inválidos em métricas numéricas são exportados como 0, com contagens garantidas como inteiros não negativos, e data_coleta registrada para rastreabilidade.
- Critério único de outliers em todas as RQs numéricas: método de Tukey ($Q1 - 1,5 \times IQR$ / $Q3 + 1,5 \times IQR$), para manter a análise comparável entre métricas.

3.3 Etapas

O trabalho foi dividido em 3 sprints, com 26 Issues concluídas no total no GitHub Projects (todas com Assignee, sem cartões draft). Distribuição por integrante: alvimdev — 8 issues; pedroseabra27 — 9 issues; luisajardim — 9 issues.

Sprint	Integrante	Entregas	Issues (nº)
SPRINT 1	Bernardo	Planejamento da arquitetura e esqueleto do Algoritmo Minerador; extração e validação inicial de RQ02/RQ03 (PRs aceitas e releases); autenticação, estrutura do projeto e exportação para CSV.	#7, #2, #1
	Pedro	Query GraphQL principal com tratamento de rate limit/erros; RQ05/RQ06 e preparo da RQ07.	#3, #4
	Luísa	README, Kanban e detalhamento das issues do repositório; extração e validação inicial de RQ01/RQ04 (idade e última atualização).	#6, #5
SPRINT 2	Pedro	Script de snapshot de fechamento de sprint do GitHub Projects (requisito 6); formulação das hipóteses informais de RQ05, RQ06 e RQ07; validação dos dados de linguagem (RQ05), da razão de issues (RQ06) e definição da fonte externa (Octoverse).	#14, #16, #15
	Luísa	Execução da extração completa e salvamento do dataset final (1.000 repositórios); formulação das hipóteses informais de RQ01 e RQ04; validação de consistência e outliers de RQ01 e RQ04.	#8, #10, #9
	Bernardo	Ajuste e padronização das colunas numéricas de PRs e releases no exportador CSV; validação de consistência e outliers de RQ02 e RQ03; formulação das hipóteses informais de RQ02 e RQ03.	#11, #12, #13
SPRINT 3	Pedro	Snapshot final do GitHub Projects v2 e integração no painel; implementação da visualização do cruzamento de dados da RQ07 e renderização dos gráficos; componentes visuais interativos em Svelte (gráficos de barra/pizza para linguagens e razões de issues).	#19, #18, #17
	Bernardo	Módulo de transformação/parse de dados CSV → JSON para o front-end Svelte; elaboração da árvore GQM; análise estatística de engajamento e releases (RQ02/RQ03).	#23, #25, #24
	Luísa	Script de análise estatística (medianas/quartis); módulo de cálculo/validação de métricas de	#20, #21, #22

Sprint	Integrante	Entregas	Issues (nº)
		maturidade (RQ01/RQ04); consolidação e formatação final do relatório.	

A divisão por integrante reflete os Assignees reais das Issues do board (GitHub Projects), com todas as 26 issues do backlog concluídas.

Configuração do processo (GitHub Projects)

O board do grupo é um GitHub Projects (v2) vinculado ao repositório luisajardim/Lab-Experimentacao-e-Medicao-. Cartões devem ser Issues com Assignee (não draft solto). Colunas mínimas de Status: Backlog do Enunciado → Backlog da Sprint → Em Andamento → Concluído.

Ao final de cada sprint, o snapshot de fechamento (requisito 6) é gerado com "npm run snapshot", executado em Enunciado 1/Miner. O script consulta o Project v2 via GraphQL (sem biblioteca cliente da API do GitHub), pagina de 100 em 100 e grava CSV em Miner/data/snapshots/, além de acumular as linhas em Miner/data/snapshots/history.csv. Variáveis de ambiente: GITHUB_PROJECT_OWNER, GITHUB_PROJECT_NUMBER, GITHUB_PROJECT_OWNER_TYPE, SPRINT_ID. O PAT precisa do escopo read:project além da leitura de repositórios públicos.

Title	...	Assignees	...	Status	...	Linked pull requests	...	Sub-issues progress	...
1	🔗 Planejar arquitetura e estruturar esqueleto do Algoritmo Minerador #7	alvimdev		Concluído					
2	🔗 Snapshot final do GitHub Projects v2 e integração no painel #19	pedroseabra27		Concluído					
3	🔗 Implementação da visualização do cruzamento de dados da RQ07 e renderização dos g... #18	pedroseabra27		Concluído					
4	🔗 Desenvolvimento de componentes visuais interativos em Svelte para exibição das RQs (...) #17	pedroseabra27		Concluído					
5	🔗 Script para Snapshot de Fechamento de Sprint do GitHub Projects (Requisito 6) #14	pedroseabra27		Concluído					
6	🔗 Formulação de Hipóteses Informais para RQ05, RQ06 e RQ07 #16	pedroseabra27		Concluído					
7	🔗 Validação dos Dados de Linguagens (RQ05), Razão de Issues (RQ06) e Definição da Fon... #15	pedroseabra27		Concluído					
8	🔗 Execução e extração dos 1.000 repositórios com salvamento no dataset data/1000_popul... #8	luisajardim		Concluído					
9	🔗 Formulação de Hipóteses Informais para RQ01 e RQ04 #10	luisajardim		Concluído					
10	🔗 Validação de Consistência e Outliers dos Dados da RQ01 (Idade em anos/dias) e RQ04 (...) #9	luisajardim		Concluído					
11	🔗 Query GraphQL principal + tratamento de rate limit/erros #3	pedroseabra27		Concluído					

12	🔗 RQ05 (linguagem primária) + RQ06 (% issues fechadas) + preparo da RQ07 #4	pedroseabra27		Concluído					
13	🔗 README do projeto+ KANBAN+ DETALHAMENTO DE ISSUES #6	luisajardim		Concluído					
14	🔗 Extração e validação: RQ01 (idade) + RQ04 (última atualização) #5	luisajardim		Concluído					
15	🔗 Extração e validação: RQ02 (PRs aceitas) + RQ03 (releases) #2	alvimdev		Concluído					
16	🔗 Autenticação, estrutura do projeto e exportação para CSV #1	alvimdev		Concluído					
17	🔗 Ajuste e padronização na gravação das colunas numéricas de PRs e Releases no export... #11	alvimdev		Concluído					
18	🔗 Validação de Consistência e Outliers dos Dados da RQ02 (PRs Aceitas) e RQ03 (Releases) #12	alvimdev		Concluído					
19	🔗 Formulação de Hipóteses Informais para RQ02 e RQ03 #13	alvimdev		Concluído					
20	🔗 Desenvolvimento do módulo de transformação/parse de dados CSV -> JSON para cons... #23	alvimdev		Concluído					
21	🔗 Elaborar Árvore GQM (Goal-Questions-Metrics) #25	alvimdev		Concluído					
22	🔗 Desenvolvimento do Script de Análise Estatística e Cálculo de Medianas/Quartis #20	luisajardim		Concluído					

23	🔗 Módulo de Cálculo e Validação de Métricas de Maturidade (RQ01 - Idade e RQ04 - Dias... #21	luisajardim		Concluído					
24	🔗 Análise Estatística de Engajamento e Releases (RQ02 - PRs Aceitas e RQ03 - Total de Rel... #24	alvimdev		Concluído					
25	🔗 Consolidação e Formatação Final do Relatório Markdown #22	luisajardim		Concluído					

3.4 Ferramentas

- API GraphQL do GitHub, consultada a partir de especificações declarativas (arquivos .yaml em Miner/specs/) e consumida por um script próprio do grupo (subprojeto Miner,

TypeScript) — sem biblioteca de terceiros para acessar a API, conforme exigido no enunciado.

- Scripts npm do subprojeto Miner: npm run dev (coleta por spec), npm run analyze (estatísticas consolidadas: mediana, quartis, outliers), npm run report:rq01-rq04, npm run validate:rq05 (validação do CSV).
- GitHub Octoverse 2025 (<https://octoverse.github.com/>) como fonte de referência de popularidade de linguagens de programação (RQ05/RQ07).
- GitHub Projects (v2) como ferramenta de processo — board do grupo vinculado ao repositório luisajardim/Lab-Experimentacao-e-Medicao-, com snapshots via GraphQL (npm run snapshot).

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ01	idade_anos	Diferença entre createdAt e a data da coleta	Anos	Script GraphQL (Miner) — github-rq1-rq4-v1.yaml
RQ02	total_pr_aceitas	pullRequests(states: MERGED).totalCount	Contagem	Script GraphQL (Miner) — github-rq2-rq3-v2.yaml
RQ03	total_releases	releases.totalCount	Contagem	Script GraphQL (Miner) — github-rq2-rq3-v2.yaml
RQ04	dias_desde_ultima_atualizacao	Diferença entre pushedAt e a data da coleta	Dias	Script GraphQL (Miner) — github-rq1-rq4-v1.yaml
RQ05	primaryLanguage + pct_no_top5_octoverse	Linguagem primária declarada; % de repositórios cuja linguagem está no top 5 do Octoverse 2025	Categoria / %	Script GraphQL (Miner) — github-search-v1.yaml; GitHub Octoverse 2025
RQ06	ratio_closed_issues	closed_issues / total_issues (0 se total_issues = 0)	Razão [0,1]	Script GraphQL (Miner) — github-search-v1.yaml
RQ07	prs_por_linguagem / releases_por_linguagem / dias_por_linguagem	Mediana de total_pr_aceitas, total_releases e dias_desde_ultima_atualizacao, agrupada por linguagem	Mediana (por grupo)	Cruzamento pós-coleta sobre o CSV unificado

A árvore GQM completa (Goals → Questions → Metrics), incluindo as submétricas de apoio (faixas de classificação e contagem de outliers por RQ), está documentada no repositório do grupo.

3.6 Inovações Propostas pelo Grupo (30% da nota)

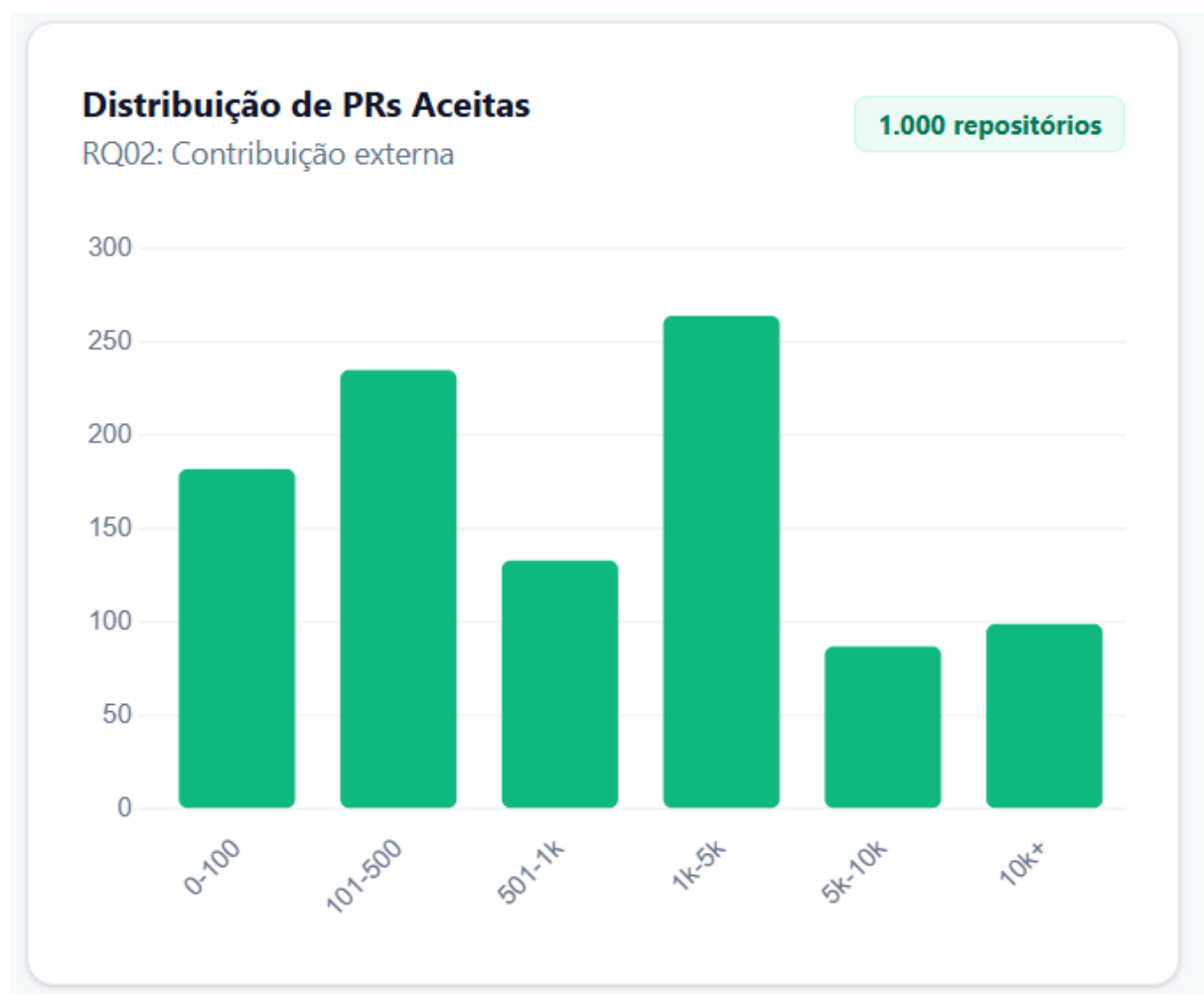
O projeto adotou uma abordagem genérica para mineração de dados, utilizando uma arquitetura análoga à hexagonal e separando implementação de script de coleta das especificações de coleta e, também, do orquestrador de implementações e de especificações distintas.

4. Resultados

4.1 Coleta de Dados

- RQ01/RQ04: 1.000 repositórios, 0 valores ausentes em `idade_anos` e em `dias_desde_ultima_atualizacao`.
- RQ02/RQ03: 1.000 registros válidos auditados, 0 valores inválidos em `total_pr_aceitas`.
- RQ05/RQ06: dataset de 1.000 repositórios, com 87 sem linguagem definida (N/A) e 43 sem issues habilitadas (`total_issues = 0`).
- Outliers foram identificados pelo critério de Tukey em todas as RQs numéricas, mas mantidos na amostra e discutidos à parte (não removidos) — ver seção 4.3.
- RQ07: extração já concentrada no CSV unificado (`linguagem`, `merged_prs`, `releases`, `dias_desde_ultima_atualizacao`); o cruzamento estatístico por linguagem estava planejado para a S03.

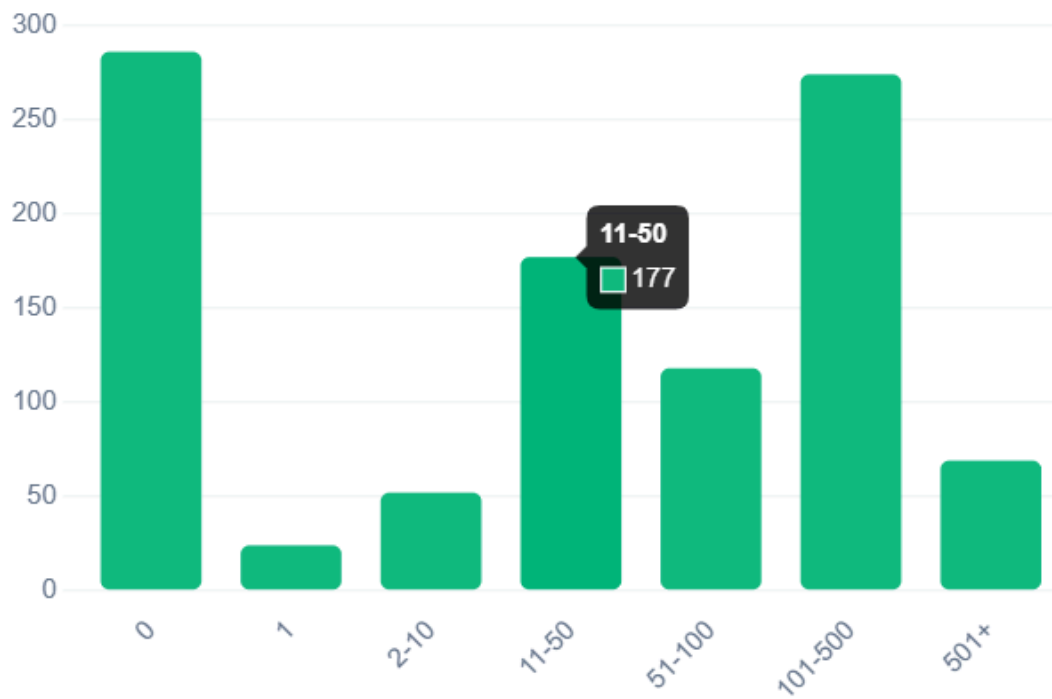
4.2 Visualização Gráfica



Distribuição de Releases

RQ03: Frequência de releases

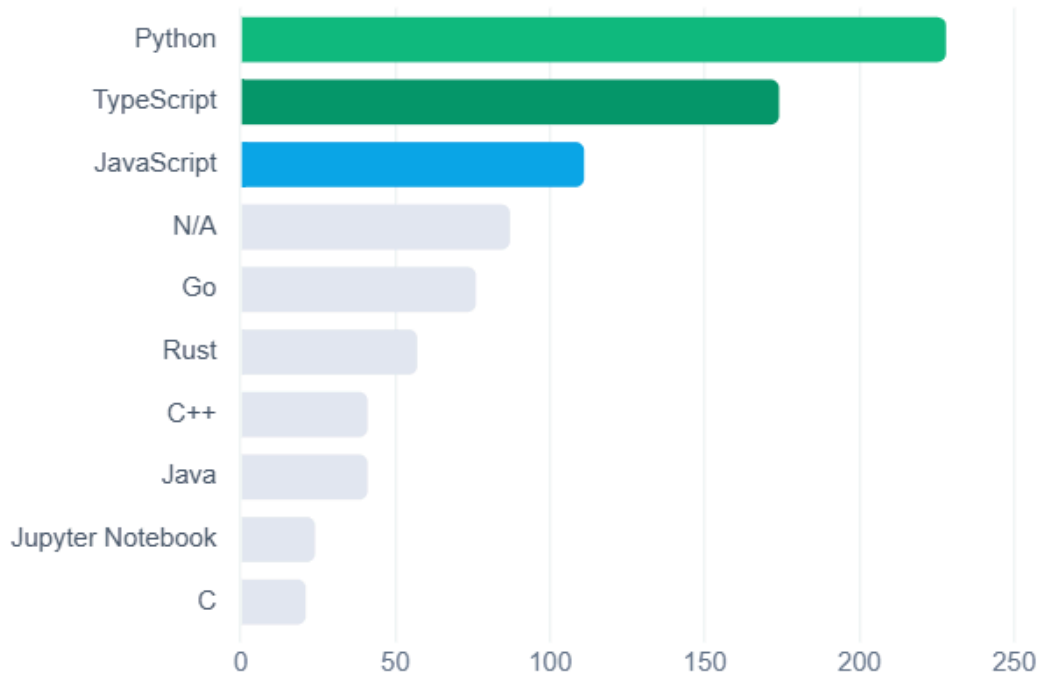
1.000 repositórios



Distribuição de Linguagens

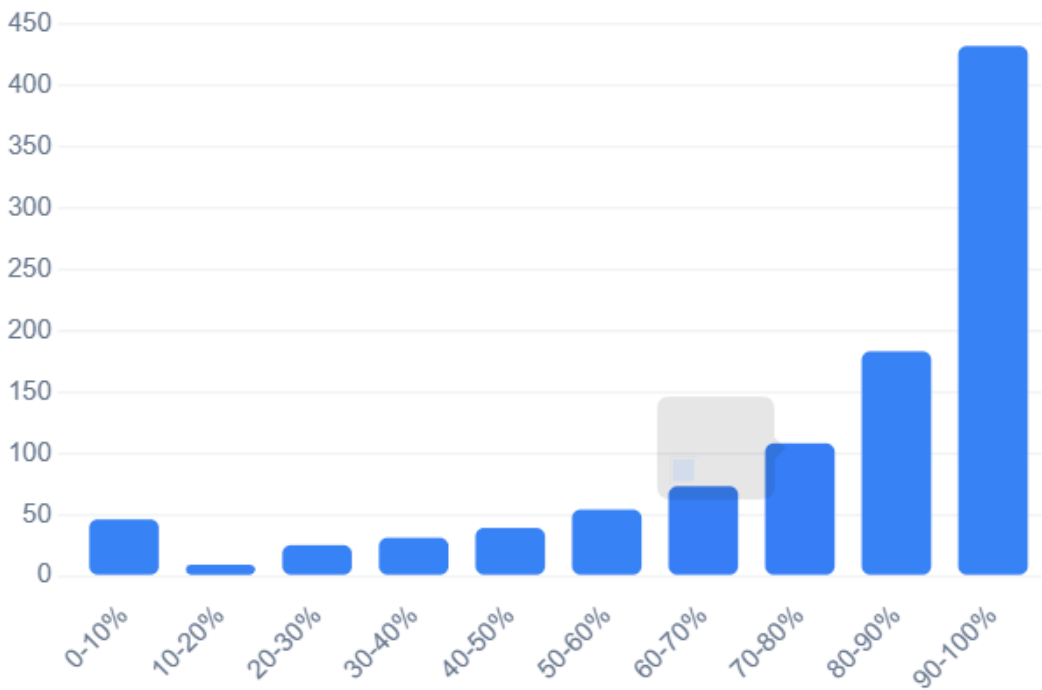
RQ05: Linguagens mais populares

Top 10



Razão de Issues Fechadas

RQ06: Distribuição de status (Histograma)



Linguagem vs Métricas

RQ07: Correlação cruzada para o top 5 linguagens

LINGUAGEM	REPOSITÓRIOS	PRS (MEDIANA)	RELEASES (MEDIANA)	IDADE (MEDIANA)
Python	228	559.5	19.5	3.9 anos
TypeScript	174	2002	133.5	6.6 anos
JavaScript	111	617	39	10.0 anos
N/A	87	129	0	9.0 anos
Go	76	1703	141	8.4 anos

RQ01 — Idade dos repositórios

Métrica	Valor
Repositórios	1.000 (0 valores ausentes)
Mínimo	0,02 anos
Q1	3,50 anos

Mediana	7,74 anos
Q3	11,35 anos
Máximo	18,36 anos
Média	7,66 anos
Faixa etária	Repositórios
até 1 ano	82 (8,2%)
1-2 anos	57 (5,7%)
2-5 anos	185 (18,5%)
5-10 anos	331 (33,1%)
10-15 anos	296 (29,6%)
15+ anos	49 (4,9%)

Critério de Tukey (IQR = 7,85 anos): faixa de aceitação de aproximadamente -8,3 a 23,1 anos — 0 outliers.

RQ02 — Contribuição externa (PRs aceitas)

Métrica	Valor
Repositórios auditados	1.000
Mínimo	0
Q1	175
Mediana	768
Q3	3.415,75
Média	4.236,77
Máximo	103.349
Outliers (Tukey)	124 (IQR = 3.240,75; limite superior $\approx$ 8.276,88)

Maiores valores concentrados em projetos com comunidades muito grandes: first-contributions, llvm-project, elasticsearch, sentry, home-assistant/core.

RQ03 — Frequência de releases

Métrica	Valor
Mínimo	0
Q1	0
Mediana	39
Q3	147
Média	126,61
Máximo	1.000

Métrica	Valor
Sem releases	286 (28,6%)
Outliers (Tukey)	93 (IQR = 147; limite superior $\approx$ 367,50)

RQ04 — Frequência de atualização

Métrica	Valor
Repositórios	1.000 (0 valores ausentes)
Mínimo	0 dias
Q1	0 dias
Mediana	2 dias
Q3	48,25 dias
Máximo	2.452 dias (~6,7 anos)
Média	113,8 dias
Faixa (dias sem push)	Repositórios
até 1 dia	477 (47,7%)
2-7 dias	132 (13,2%)
8-30 dias	118 (11,8%)
31-90 dias	64 (6,4%)
91-365 dias	94 (9,4%)
mais de 365 dias	115 (11,5%)

Limite superior de Tukey $\approx$ 120,6 dias — 196 outliers (19,6%). Casos mais extremos: exacity/deeplearningbook-chinese (2.452 dias), GitSquared/edex-ui (1.765), lib-pku/libpku (1.688), adobe/brackets (1.529), atom/atom (1.324).

RQ05 — Linguagem primária

Linguagem	Contagem	%
Python	227	22,7%
TypeScript	174	17,4%
JavaScript	111	11,1%
Sem linguagem (N/A)	87	8,7%
Go	76	7,6%
Rust	57	5,7%
C++	41	4,1%
Java	41	4,1%

Nas 5 linguagens do Octoverse 2025 (TypeScript, Python, JavaScript, Java, C#): 561 repositórios (56,1%).

RQ06 — Percentual de issues fechadas

Métrica	Valor
Razão mínima	0,00
Razão mediana	0,8649 (86,5%)
Razão máxima	1,00
Repositórios sem issues	43
Razão fora de [0,1]	0

RQ07 — Cruzamento por linguagem

Abaixo são apresentadas as medianas das métricas de contribuição, versionamento e atualização para as linguagens mais populares da amostra.

Linguagem	Repositórios	PRs aceitas (mediana)	Releases (mediana)	Dias desde o último push (mediana)
TypeScript	174	2.002,5	133,5	0
Python	227	559	20	2
JavaScript	111	617	39	5
Java	41	943	55	2
C#	8	3.116	126	0,5

4.3 Discussão

RQ01 — A hipótese se confirma. Mediana em 7,74 anos e média próxima (7,66) indicam distribuição sem grande distorção, com 62,7% dos repositórios entre 5 e 15 anos. Apenas 13,9% têm menos de 2 anos, reforçando que popularidade no GitHub costuma levar tempo. Como o critério de Tukey não aponta nenhum outlier em idade_anos, os repositórios jovens não são uma exceção estatística — são a ponta mais nova de uma distribuição contínua, puxada provavelmente pela onda de ferramentas de IA/agentes de código surgida em 2025-2026.

RQ02 — A hipótese se confirma, com uma ressalva: a distribuição é extremamente desigual. A mediana de 768 PRs aceitos mostra um fluxo constante de contribuição na maioria dos projetos, enquanto os valores extremos (até 103.349) puxam a média para 4.236,77 — típico de projetos muito grandes com unidades de manutenção dedicadas. Contribuição externa parece ser a regra, mas a intensidade varia muito entre repositórios.

RQ03 — A hipótese se sustenta parcialmente. A mediana de 39 releases e o Q3 em 147 indicam uso regular de versionamento por parte relevante da amostra, mas 28,6% dos projetos não usam a funcionalidade Releases do GitHub, sugerindo distribuição por meios alternativos (tags, pacotes, imagens de contêiner). Como o total acumulado favorece repositórios mais antigos, a leitura mais confiável é a mediana/distribuição, não os máximos absolutos.

RQ04 — A hipótese se confirma, mas o segundo grupo previsto (repositórios populares sem atualização recente) tem um perfil diferente do imaginado. A mediana de 2 dias mostra atividade quase diária em quase metade da amostra (47,7%), mais frequente que o estimado originalmente. Já

a média (113,8 dias), bem acima da mediana, denuncia uma cauda longa de projetos abandonados (os mesmos 19,6% de outliers) — majoritariamente ferramentas de software oficialmente descontinuadas pelos mantenedores (atom/atom, adobe/brackets), e não listas/material estático como previa a hipótese original.

RQ05 — A hipótese se sustenta em parte. Python, TypeScript e JavaScript são de fato as três linguagens primárias mais frequentes, alinhadas ao topo do Octoverse 2025, mas a ordem local não copia o ranking de contribuidores (Python lidera a amostra com 22,8%, enquanto o Octoverse coloca TypeScript em 1º). Os 8,7% sem linguagem definida (listas awesome-*, materiais) puxam o cruzamento para baixo se não tratados à parte.

RQ06 — A hipótese se confirma: a mediana de ~86,5% supera o limiar de 70% esperado. Os 43 repositórios com razão 0, por não usarem a aba Issues (ex.: torvalds/linux e listas), puxam o mínimo para 0 mas não derrubam a mediana. Não houve inconsistência do tipo `closed_issues > total_issues` no CSV.

RQ07 — A hipótese se confirma para linguagens como TypeScript e C#, que apresentam as maiores medianas de PRs aceitos e releases, além de atividade recente imediata (mediana de 0 e 0,5 dias). Python e Java apresentam engajamento moderado, enquanto JavaScript demonstra uma frequência de atualização ligeiramente menor em comparação aos líderes do ranking.

As três limitações do laboratório (fonte única do Octoverse 2025 para popularidade de linguagem, coleta em janela única de tempo e critério de outliers uniforme entre métricas distintas) constituem escolhas metodológicas deliberadas e mantidas de forma consistente ao longo de todo o estudo, não falhas que comprometam as respostas às RQs. A fonte única evita misturar rankings incomparáveis, a janela única é coerente com a proposta de um estudo de caracterização transversal, e o critério padronizado de outliers torna a análise comparável entre métricas, sem que nenhuma conclusão obtida tenha sido revertida por esse critério.

A inovação apresentada na seção 3.6, referente à arquitetura análoga à hexagonal que separa script de coleta, especificações e orquestrador, não elimina essas limitações de forma retroativa nos dados já coletados. Entretanto, por desacoplar a extração da fonte e do protocolo utilizados, ela reduz o custo de endereçar tais limitações em trabalhos futuros, possibilitando testar novas fontes de popularidade de linguagem, repetir a coleta em outras janelas de tempo ou aplicar critérios de outliers alternativos para cada métrica.

5. Conclusão

As sete RQs foram respondidas descritivamente com base nos 1.000 repositórios do CSV oficial. As hipóteses de RQ01, RQ04 e RQ06 se confirmaram; RQ02, RQ03, RQ05 e RQ07 se confirmaram apenas parcialmente ou com ressalvas. Em conjunto, os dados mostram repositórios populares majoritariamente maduros (mediana de 7,74 anos), ativos (mediana de 2 dias desde o último push) e com alta razão de issues fechadas (mediana de 86%), embora contribuição externa e releases sejam fortemente desiguais entre projetos. Com mais tempo e recursos, o grupo repetiria a coleta em diferentes momentos, normalizaria PRs e releases pela idade dos repositórios, aumentaria a amostra de linguagens menos representadas e separaria releases do GitHub de tags, pacotes e imagens de contêiner. Também seria útil comparar formalmente os grupos com testes não paramétricos e intervalos de confiança. Entre as inovações propostas, vale expandir o minerador modular baseado em especificações declarativas, o dashboard interativo e o sistema de snapshots do GitHub Projects, tornando o estudo transversal uma série histórica.

6. Referências

- GITHUB. Octoverse 2025. Disponível em: <https://octoverse.github.com/>. Acesso em: ago. 2025.
- BASILI, V. R.; CALDIERA, G.; ROMBACH, H. D. The Goal Question Metric Approach. Encyclopedia of Software Engineering, 1994.